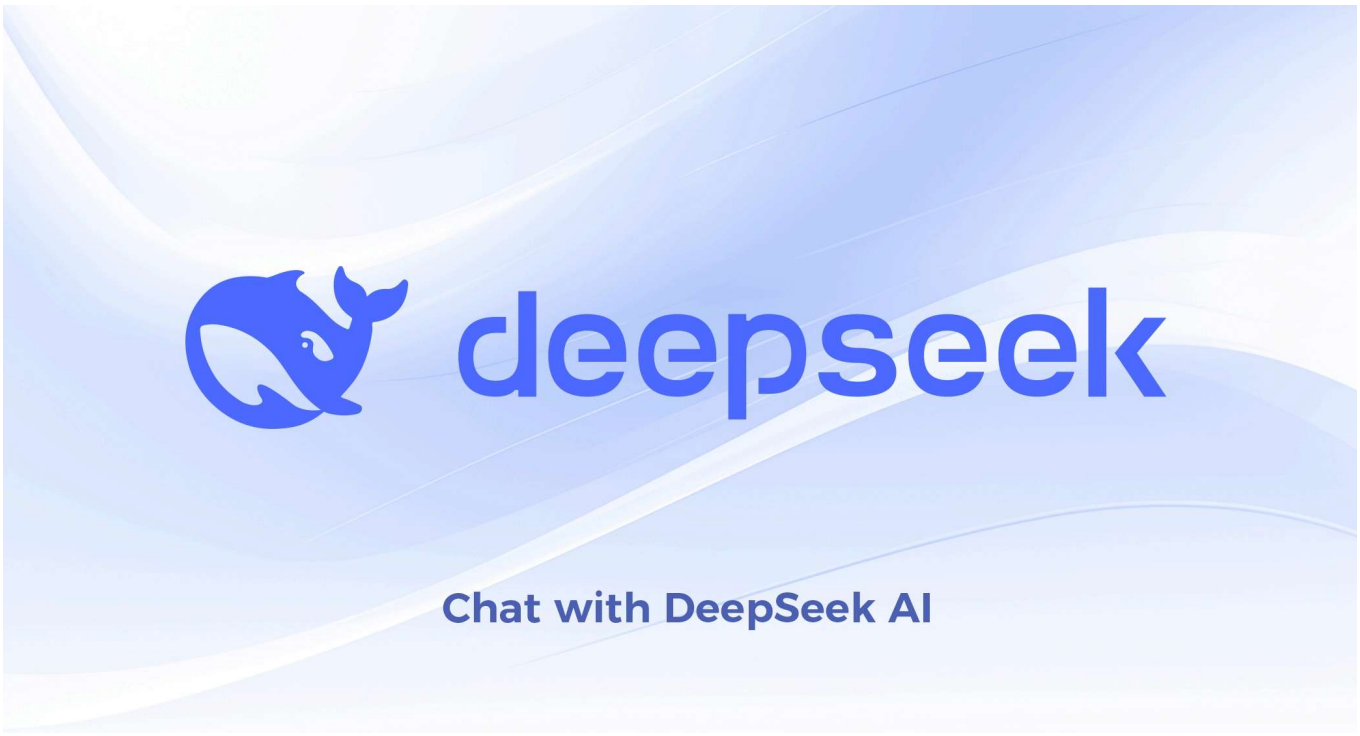
Bitwise Operators Guide - DeepSeek



bitwise operators in python and all detailed explanation with example and code also all type 1 to 10 binary to decimal decimal to binary

Bitwise Operators in Python - Complete Guide

Table of Contents

1. Introduction to Bitwise Operators
2. Binary Number System
3. Bitwise Operators in Python
4. Binary to Decimal Conversion
5. Decimal to Binary Conversion
6. Complete Examples

1. Introduction to Bitwise Operators {#introduction}

Bitwise operators are used to perform operations on individual bits of integers. These operators work on binary representations of numbers at the bit level.

2. Binary Number System {#binary-system}

Binary is a base-2 number system that uses only two digits: 0 and 1.

Binary to Decimal Conversion (1 to 10)

3. Bitwise Operators in Python {#operators}

Python provides six bitwise operators:

3.1 Bitwise AND (&)

Returns 1 if both bits are 1, otherwise 0.

Truth Table:

text

```
A | B | A & B
0 | 0 |   0
0 | 1 |   0
1 | 0 |   0
1 | 1 |   1
```

python

```
a = 12
b = 10

result = a & b
print(f"{a} & {b} = {result}")
print(f"Binary: {bin(a)} & {bin(b)} = {bin(result)}")
print(f"  1100")
print(f"& 1010")
print(f"  ----")
print(f"  1000 = 8")
```

3.2 Bitwise OR (|)

Returns 1 if at least one bit is 1.

Truth Table:

text

```
A | B | A | B
0 | 0 |   0
0 | 1 |   1
1 | 0 |   1
1 | 1 |   1
```

python

```
a = 12
b = 10

result = a | b
print(f"{a} | {b} = {result}")
print(f"Binary: {bin(a)} | {bin(b)} = {bin(result)}")
print(f"  1100")
print(f"| 1010")
print(f"  ----")
print(f"  1110 = 14")
```

3.3 Bitwise XOR (^)

Returns 1 if bits are different, 0 if same.

Truth Table:

text

```
A | B | A ^ B
0 | 0 |   0
0 | 1 |   1
1 | 0 |   1
1 | 1 |   0
```

python

```
a = 12
b = 10

result = a ^ b
print(f"{a} ^ {b} = {result}")
print(f"Binary: {bin(a)} ^ {bin(b)} = {bin(result)}")
print(f"  1100")
print(f"^ 1010")
print(f"  ----")
print(f"  0110 = 6")
```

3.4 Bitwise NOT (~)

Inverts all bits (one's complement).

python

```
a = 10

result = ~a
print(f"~{a} = {result}")
print(f"Binary: ~{bin(a)} = {bin(result)}")
print(f"  00001010 (10 in 8-bit)")
print(f"~ 11110101 (inverted)")
print(f"  In Python: ~10 = -11")
```

3.5 Left Shift (<<)

Shifts bits to the left, filling with zeros.

python

```
a = 5

print(f"{a} << 1 = {a << 1}")
print(f"{a} << 2 = {a << 2}")
print(f"{a} << 3 = {a << 3}")

print("\nVisual Representation:")
print(f"5 in binary:  {bin(a)}")
print(f"5 << 1:       {bin(a << 1)} = {a << 1}")
print(f"5 << 2:       {bin(a << 2)} = {a << 2}")
print(f"5 << 3:       {bin(a << 3)} = {a << 3}")
```

3.6 Right Shift (>>)

Shifts bits to the right.

python

```
a = 32

print(f"{a} >> 1 = {a >> 1}")
print(f"{a} >> 2 = {a >> 2}")
print(f"{a} >> 3 = {a >> 3}")

print("\nVisual Representation:")
print(f"32 in binary: {bin(a)}")
print(f"32 >> 1:      {bin(a >> 1)} = {a >> 1}")
print(f"32 >> 2:      {bin(a >> 2)} = {a >> 2}")
print(f"32 >> 3:      {bin(a >> 3)} = {a >> 3}")
```

4. Binary to Decimal Conversion {#binary-decimal}

Method 1: Using int() function

python

```
def binary_to_decimal_using_int(binary_string):
    """Convert binary string to decimal using int()"""
    return int(binary_string, 2)


binary_numbers = ['0001', '0010', '0011', '0100', '0101',
                  '0110', '0111', '1000', '1001', '1010']

print("Binary to Decimal (1-10):")
print("-" * 30)
for binary in binary_numbers:
    decimal = binary_to_decimal_using_int(binary)
    print(f"Binary: {binary} = Decimal: {decimal}")
```

Method 2: Manual Conversion

python

```
def binary_to_decimal_manual(binary_string):
    """Convert binary string to decimal manually"""
    decimal = 0
    power = 0


    for digit in reversed(binary_string):
        if digit == '1':
            decimal += 2 ** power
        power += 1

    return decimal


print("\nManual Binary to Decimal Conversion:")
print("-" * 40)
for i in range(1, 11):
    binary = bin(i)[2:].zfill(4)
    decimal = binary_to_decimal_manual(binary)
    print(f"Binary: {binary} = Decimal: {decimal}")
```

5. Decimal to Binary Conversion {#decimal-binary}

Method 1: Using bin() function

python

```
def decimal_to_binary_using_bin(decimal):
    """Convert decimal to binary using bin()"""
    return bin(decimal)[2:]


print("Decimal to Binary (1-10):")
print("-" * 30)
for i in range(1, 11):
    binary = decimal_to_binary_using_bin(i)
    print(f"Decimal: {i:2d} = Binary: {binary}")
```

Method 2: Manual Conversion

python

```
def decimal_to_binary_manual(decimal):
    """Convert decimal to binary manually"""
    if decimal == 0:
        return "0"

    binary = ""
    while decimal > 0:
        binary = str(decimal % 2) + binary
        decimal = decimal // 2

    return binary


print("\nManual Decimal to Binary Conversion:")
print("-" * 40)
for i in range(1, 11):
    binary = decimal_to_binary_manual(i)
    print(f"Decimal: {i:2d} = Binary: {binary}")
```

6. Complete Examples {#complete-examples}

Example 1: Bit Manipulation Operations

python

```
def bit_manipulation_demo():
    """Demonstrate various bit manipulation operations"""
    print("Bit Manipulation Demo")
    print("=" * 50)

    def set_bit(num, position):
        return num | (1 << position)

    def clear_bit(num, position):
        return num & ~(1 << position)

    def toggle_bit(num, position):
        return num ^ (1 << position)

    def check_bit(num, position):
        return (num & (1 << position)) != 0

    num = 10
    print(f"Original number: {num} ({bin(num)})")

    result = set_bit(num, 2)
    print(f"After setting bit 2: {result} ({bin(result)})")

    result = clear_bit(num, 1)
    print(f"After clearing bit 1: {result} ({bin(result)})")

    result = toggle_bit(num, 0)
    print(f"After toggling bit 0: {result} ({bin(result)})")

    print(f"Bit at position 3 is set: {check_bit(num, 3)}")
```

Example 2: Even/Odd Check using Bitwise

python

```
def check_even_odd(num):
    """Check if number is even or odd using bitwise AND"""
    if num & 1:
        return "Odd"
    else:
        return "Even"


print("Even/Odd Check using Bitwise:")
print("-" * 30)
for i in range(1, 11):
    result = check_even_odd(i)
    print(f"{i:2d} is {result}")
```

Example 3: Multiplication and Division by 2

python

```
def multiply_divide_by_2():
    """Demonstrate multiplication and division using shift operators"""
    print("\nMultiplication/Division by 2 using Shifts:")
    print("-" * 40)

    for i in range(1, 6):

        multiplied = i << 1

        divided = i >> 1

        print(f"{i} * 2 = {multiplied} (using << 1)")
        print(f"{i} / 2 = {divided} (using >> 1)")
        print()
```

Example 4: Swap Numbers using XOR

python

```
def swap_without_temp():
    """Swap two numbers without using temporary variable"""
    print("Swap Numbers using XOR:")
    print("-" * 30)

    a, b = 5, 9
    print(f"Before swap: a = {a}, b = {b}")


    a = a ^ b
    b = a ^ b
    a = a ^ b

    print(f"After swap: a = {a}, b = {b}")
```

Example 5: Power of 2 Check

python

```
def is_power_of_two(num):
    """Check if number is power of 2 using bitwise"""
    return num > 0 and (num & (num - 1)) == 0


print("\nPower of 2 Check:")
print("-" * 30)
for i in range(1, 17):
    if is_power_of_two(i):
        print(f"{i:2d} is a power of 2")
```

Example 6: Count Set Bits

python

```
def count_set_bits(num):
    """Count number of set bits (1s) in binary representation"""
    count = 0
    while num:
        count += num & 1
        num >>= 1
    return count


print("\nCount Set Bits:")
print("-" * 30)
for i in range(1, 11):
    binary = bin(i)[2:]
    set_bits = count_set_bits(i)
    print(f"{i:2d} ({binary:4s}) has {set_bits} set bit(s)")
```

Complete Program

python

```
def main():
    """Main function to run all examples"""
    print("BITWISE OPERATORS COMPLETE GUIDE")
    print("=" * 60)


    a, b = 12, 10
    print(f"\nBasic Operations with a={a} ({bin(a)}) and b={b} ({bin(b)}):")
    print(f"a & b  = {a & b} ({bin(a & b)})")
    print(f"a | b  = {a | b} ({bin(a | b)})")
    print(f"a ^ b  = {a ^ b} ({bin(a ^ b)})")
    print(f"~a     = {~a} ({bin(~a)})")
    print(f"a << 2 = {a << 2} ({bin(a << 2)})")
    print(f"a >> 2 = {a >> 2} ({bin(a >> 2)})")


    print("\n" + "=" * 60)
    print("CONVERSION EXAMPLES")
    print("=" * 60)


    print("\nDecimal to Binary:")
    for i in range(1, 11):
        print(f"{i:2d} = {bin(i)[2:]:4s}")


    print("\nBinary to Decimal:")
    for i in range(1, 11):
        binary = bin(i)[2:].zfill(4)
        print(f"{binary} = {i:2d}")


    bit_manipulation_demo()
    check_even_odd(10)
    multiply_divide_by_2()
    swap_without_temp()
    is_power_of_two(16)
    count_set_bits(15)

if __name__ == "__main__":
    main()
```

Summary Table of Bitwise Operators

This comprehensive guide covers all bitwise operators in Python with detailed explanations, examples, and conversion methods between binary and decimal systems.

table format

Bitwise Operators in Python - Complete Guide

1. Binary to Decimal Conversion Table (1 to 10)

2. Bitwise Operators Summary Table

3. Truth Tables for Bitwise Operators

AND Truth Table

OR Truth Table

XOR Truth Table

NOT Truth Table

4. Detailed Bitwise Operations with Examples

Bitwise AND (&) Operation

python

```
a = 12
b = 10
result = a & b
print(f"{a} & {b} = {result}")
```

Bitwise OR (|) Operation

python

```
a = 12
b = 10
result = a | b
print(f"{a} | {b} = {result}")
```

Bitwise XOR (^) Operation

python

```
a = 12
b = 10
result = a ^ b
print(f"{a} ^ {b} = {result}")
```

5. Shift Operations Table

Left Shift (<<)

Right Shift (>>)

6. Common Bit Manipulation Techniques Table

7. Conversion Methods Table

8. Complete Code Examples

python

```
print("=" * 60)
print("BITWISE OPERATORS COMPLETE DEMONSTRATION")
print("=" * 60)


print("\n1. BASIC BITWISE OPERATIONS")
print("-" * 40)
a, b = 12, 10
print(f"a = {a} (binary: {bin(a)})")
print(f"b = {b} (binary: {bin(b)})")
print(f"a & b  = {a & b} (binary: {bin(a & b)})")
print(f"a | b  = {a | b} (binary: {bin(a | b)})")
print(f"a ^ b  = {a ^ b} (binary: {bin(a ^ b)})")
print(f"~a     = {~a} (binary: {bin(~a)})")
print(f"a << 2 = {a << 2} (binary: {bin(a << 2)})")
print(f"a >> 2 = {a >> 2} (binary: {bin(a >> 2)})")


print("\n2. CONVERSION FUNCTIONS")
print("-" * 40)

def decimal_to_binary(decimal):
    return bin(decimal)[2:]

def binary_to_decimal(binary_str):
    return int(binary_str, 2)


print("\nDecimal to Binary (1-10):")
print("-" * 30)
print("Decimal | Binary")
print("-" * 20)
for i in range(1, 11):
    print(f"{i:7d} | {decimal_to_binary(i):6s}")


print("\nBinary to Decimal (1-10):")
print("-" * 30)
print("Binary | Decimal")
print("-" * 20)
for i in range(1, 11):
    binary = decimal_to_binary(i).zfill(4)
    print(f"{binary:6s} | {binary_to_decimal(binary):7d}")


print("\n3. BIT MANIPULATION TECHNIQUES")
print("-" * 40)

def set_bit(num, position):
```

```
    return num | (1 << position)

def clear_bit(num, position):
    return num & ~(1 << position)

def toggle_bit(num, position):
    return num ^ (1 << position)

def check_bit(num, position):
    return (num & (1 << position)) != 0


num = 10
print(f"Original number: {num} (binary: {bin(num)})")
print(f"Set bit 2: {set_bit(num, 2)} (binary: {bin(set_bit(num, 2))})")
print(f"Clear bit 1: {clear_bit(num, 1)} (binary: {bin(clear_bit(num, 1))})")
print(f"Toggle bit 0: {toggle_bit(num, 0)} (binary: {bin(toggle_bit(num, 0))})")
print(f"Check bit 3: {check_bit(num, 3)}")
```

9. Performance Comparison Table